

一、简介

二、VPN

2.1 解压并安装

2.2 设置

三、服务器

四、docker

4.1 检查容器状态

4.2 进入与退出容器

五、开发环境使用方法

5.1 GPU驱动安装

5.2 容器内安装软件

5.3 测试用例

5.3.1 编译

5.3.2 执行

5.4 文件拷贝

5.4.1 本地和服务器之间复制文件

5.4.2 服务器和容器之间复制文件

一、简介

VPN -> 服务器 -> docker

每支参赛队一个VPN账号，一个服务器账号，一个docker。

Windows操作系统登录VPN，使用远程连接服务器，服务器内打开docker。

代码请运行在docker内。

二、VPN

VPN账号和密码单独通知。

2.1 解压并安装

注意：当前VPN客户端仅支持**Windows**操作系统，后续操作应保证VPN已经建立。每天**1:00到8:00 VPN关闭**，参赛选手注意保存开发代码。

解压【secportal.5.3.0.527增强版.zip】压缩文件，双击【Setup.exe】运行安装程序；

点击【一键安装】，进行自动化安装；



2.2 设置

安装完成后，打开SecPortal软件，右上方【设置】；



点击【常规设置】-->【网关设置】-->【添加】

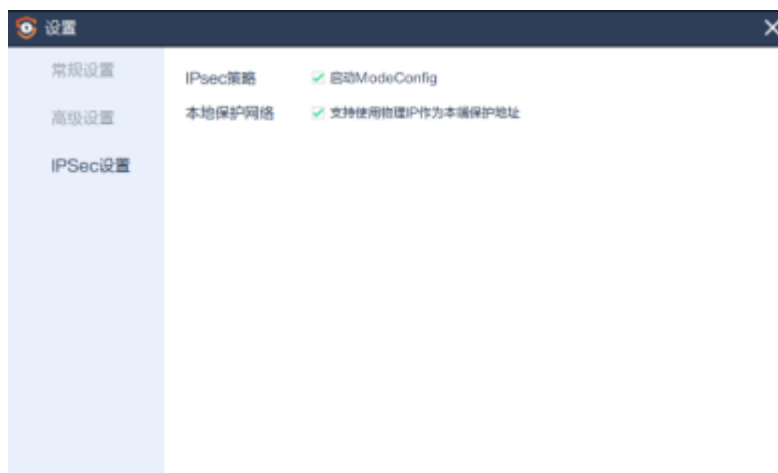


如下对认证服务器进行设置，协议选择【SSL】，地址为【58.214.247.98】，端口为【11443】；



【高级设置】及【IPSec设置】如下图所示





设置完成后，【认证方式】选择账号密码认证，输入账号密码即可登录。



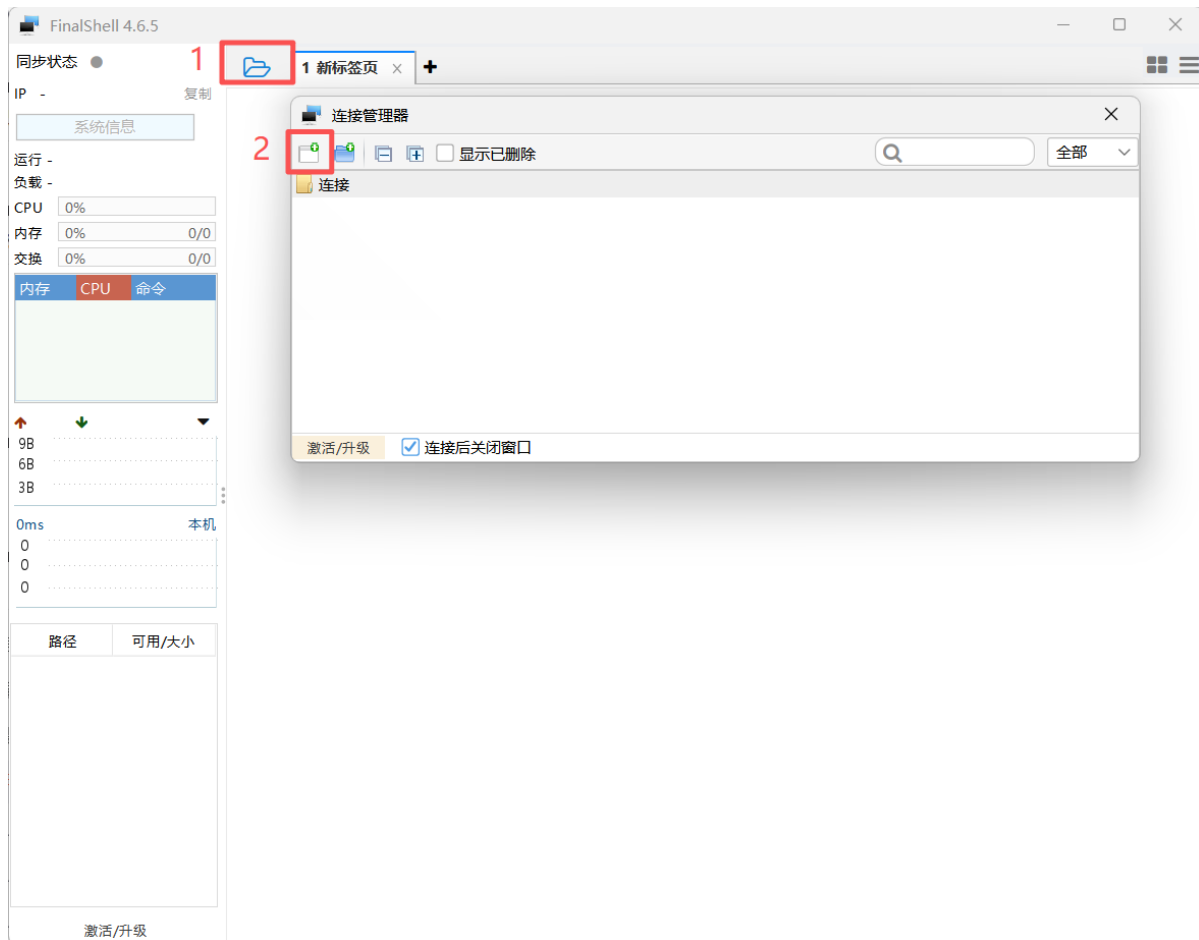
三、服务器

为每支参赛队提供一个服务器账号。账号名和初始密码与vpn一样。参赛者可以选择修改初始密码。

请仅使用自己用户路径，严禁参赛者修改公共路径和其他参赛者用户路径内容！

参赛者在Windows操作系统中可以选择任意远程连接工具连接服务器，以 FinalShell 为例，介绍SSH远程连接工具的使用。也可使用其他工具如mobaxterm、XShell等。

安装FinalShell，并新建【SSH】连接：



选中下拉列表中的SSH连接，并配置【SSH】连接：

注意：用户名与密码见章节二。

新建连接

SSH连接

终端

代理服务器

隧道

常规

名称: host

主机: 10.110.12.10 端口: 22

备注:

认证

方法: 密码

用户名: usr_00

密码:

私钥: 浏览...

高级

☐ 智能加速 (加速海外服务器连接)

☒ 启用Exec Channel(若连接上就被断开,请关闭该项,比如跳板机)

关闭后无法监控服务器信息

确定 取消

连接后显示:

FinalShell 4.6.5

同步状态

IP 10.110.12.10 复制

系统信息

运行 12:29

负载 128.98, 128.98, 129.00

CPU 0%

内存 1% 11.5G/814.6G

交换 0% 0/2G

内存	CPU	命令
6.2M	20	top
12.8M	0	systemd
0	0	kthreadd
0	0	rcu_gp

912B 360B ens2f0

3K

2K

1K

0ms

本机

路径	可用/大小
/dev	407.3G/407.3G
/run	81.5G/81.5G
/	3.2T/3.4T
/dev/shm	407.3G/407.3G
/run/lock	5M/5M
/sys/fs/cgr...	407.3G/407.3G
/snap/bare...	0/128K
/snap/core...	0/63M
/snap/sna...	0/46M
/snap/qno...	0/346M

激活/升级

1 host

* Support: <https://ubuntu.com/advantage>

* Introducing Expanded Security Maintenance for Applications.
Receive updates to over 25,000 software packages with your Ubuntu Pro subscription. Free for personal use.

<https://ubuntu.com/pro>

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`
Failed to connect to <https://changelogs.ubuntu.com/meta-release-lts>. Check your Internet connection or proxy settings

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Thu May 7 22:37:13 2026 from 192.168.220.253
usr_00@supervisor-X7650H0:~\$

命令输入 (双击Ctrl切换,Alt历史,Tab路径/命令,Esc关闭窗口)

历史 选项

文件 命令

/home/usr_00

历史

文件名 ^	大小	类型	修改时间	权限	用
.cache		文件夹	2026/05/07 15:36	drwxr-xr-x	usr_
.config		文件夹	2026/05/07 22:07	drwx-----	usr_
.gnupg		文件夹	2026/05/07 10:26	drwx-----	usr_
.local		文件夹	2026/05/07 10:26	drwxr-xr-x	usr_
.pki		文件夹	2026/05/07 11:14	drwx-----	usr_
.vscode		文件夹	2026/05/07 11:14	drwxrwxr-x	usr_
.vscode-server		文件夹	2026/05/07 21:46	drwxr-x---	usr_
.vscode-shared		文件夹	2026/05/07 11:14	drwxrwxr-x	usr_

四、docker

docker是容器化平台，每个docker内将应用及其依赖库、配置等，封装为轻量化独立隔离运行环境中，实现与宿主机和其它容器的环境隔离、资源隔离。

服务器上已经建立了32个docker，每个docker与一个GPU硬件设备绑定。每个用户仅能操作自己对应的docker。

服务器上已经为每支队伍建立的对应的docker容器，宿主机仅用于**文件中转**，参赛队伍的文件请放在**个人用户目录**下，请在本队伍对应的**docker容器内**进行代码开发与调试。

docker创建时network为host，服务器内容容器内直接共用宿主机本机IP、网卡、端口。

4.1 检查容器状态

找到自己的容器，并保证自己的容器处于 start 状态，每支参赛队伍只有操作自己容器的权限。

通过以下命令检查容器是否处于 start 状态：

查看所有容器信息：

```
sudo docker ps -a
```

或只查看运行容器的信息：

```
sudo docker ps
```

找到自己的容器，保证容器已经启动，若未启动（状态是Exited）：

```
usr_01@supervisor-X7850H0:~$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2002cb33bcf4	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_31
0d228877b570	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_30
afd32b372963	7198	"/bin/bash"	10 days ago	Exited (137) 10 days ago		gpu_29
b455a2f52b41	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_28
33ef78682e24	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_27
41c648489ebb	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_26
366f408a2d13	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_25
08660062943f	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_24
48d1d53cc842	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_23
7b876b01f923	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_22
e3537ca8b76a	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_21
c3ee2fef2552	7198	"/bin/bash"	10 days ago	Exited (137) 10 days ago		gpu_20
310810c9bfaf	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_19
3a99179555d3	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_18
c3afd259d135	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_17
a3223f4d7f57	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_16
c9f4c50cda85	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_15
2092ec9ac0c0	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_14
0936a822437a	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_13
e91b34dbf909	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_12
c9a960cf4d4e	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_11
fded730842c1	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_10
4caf669fffb2	7198	"/bin/bash"	10 days ago	Up 2 days		gpu_09
e2d20f354ed0	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_08
b597cb53f280	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_07
8ab4100d7744	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_06
fc28eba6dd86	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_05
e7f334172ec8	7198	"/bin/bash"	10 days ago	Exited (0) 10 days ago		gpu_04
b3417c6f42d1	7198	"/bin/bash"	12 days ago	Exited (0) 12 days ago		gpu_03
ada165bc32e2	7198	"/bin/bash"	12 days ago	Exited (0) 12 days ago		gpu_02
003452ebc5b8	7198	"/bin/bash"	12 days ago	Exited (137) 10 days ago		gpu_01
2e3d753cad9e	7198	"/bin/bash"	12 days ago	Up 2 days		gpu_00

```
usr_01@supervisor-X7850H0:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
4caf669fffb2	7198	"/bin/bash"	10 days ago	Up 2 days		gpu_09
2e3d753cad9e	7198	"/bin/bash"	12 days ago	Up 2 days		gpu_00

则利用start命令启动容器：

```
sudo docker start docker_name
```

```
usr_01@supervisor-X7850H0:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS       NAMES
4caf669ffffb2   7198     "/bin/bash"             10 days ago   Up 2 days                    gpu_09
2e3d753cad9e    7198     "/bin/bash"             12 days ago   Up 2 days                    gpu_00
usr_01@supervisor-X7850H0:~$ sudo docker start gpu_01
gpu_01
usr_01@supervisor-X7850H0:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS       NAMES
4caf669ffffb2   7198     "/bin/bash"             10 days ago   Up 2 days                    gpu_09
003452ebc5b8    7198     "/bin/bash"             12 days ago   Up 2 minutes                gpu_01
2e3d753cad9e    7198     "/bin/bash"             12 days ago   Up 2 days                    gpu_00
usr_01@supervisor-X7850H0:~$
```

start 状态下可关闭或者重启docker

```
sudo docker restart docker_name
```

```
sudo docker stop docker_name
```

4.2 进入与退出容器

保证自己的容器是 start 状态，并进入容器：

```
sudo docker exec -it docker_name bash
```

```
usr_01@supervisor-X7850H0:~$ sudo docker exec -it gpu_01 bash
(base) root@supervisor-X7850H0:/#
```

此时即成功进入对应的容器终端。

退出容器可通过：

```
exit
```

五、开发环境使用方法

5.1 GPU驱动安装

进入docker容器终端后，要初始化开发环境。每次进入容器都需要执行此命令。

```
source /zq500/sdk/env.sh
```

```
(base) root@supervisor-X7850H0:/# source /zq500/sdk/env.sh
Activate Release Package @ /zq500/sdk
(base) root@supervisor-X7850H0:/#
```

开发环境初始化完成后，通过以下命令查看GPU设备信息，docker已与设备绑定，此时尽可以看到绑定的设备信息：

```
dlsmi
```

```
(sdk) (base) root@supervisor-X7850H0:/# dlsmi
Thu May 7 14:40:34 2026

+-----+
| DL-SMI 12.0 | Driver version 2.3.11 |
+-----+
| GPU Name | Bus-Id | Cluster-Memory-Usage | GPU-Util |
| Fan Temp Perf Pwr:Usage/Cap | Memory-Usage | 0 1 2 3 | MIG M. |
+-----+
| Board 0 | Board-Pwr 56W / 150W |
+-----+
| 0 ZQ500-Q QUAD-0 | 0000:06:00.0 | 6% |
| 0% 68C P0 | 5W | 293 / 16384 | 293 | 0 | 0 | 0 | Disabled |
+-----+

+-----+
| Processes: |
| GPU GI | PID Process name | Cluster-Memory-Usage | GPU-Memory |
| ID | TASK | 0 1 2 3 | Usage |
+-----+
| No running processes found |
+-----+
```

如图所示，则说明GPU驱动环境正常，可正常使用。

5.2 容器内安装软件

服务器能够连通外网的时间只有5天，请尽快安装所需软件！！

```
apt update

apt install ***
```

无法连接外网后，只能使用安装包安装的方式。

5.3 测试用例

开发环境提供了大量的测试用例，供参赛者编写代码参考。

测试环境docker容器内路径为：

```
/zq500/sdk/samples/
```

下文的编译和执行请在具体测试用例路径下运行。

另外，/zq500/cuda11/include 下可以查看支持的函数。

5.3.1 编译

测试用例都是通过CMake构建的。

每个测试用例都有CMakeList.txt，参赛者可以仿照编写自己的cmake编译脚本。

```
mkdir build
cd build
cmake ..
make
```

5.3.2 执行

编译完成后build目录下会生成可执行文件

```
./可执行测试用例名
```

以 /zq500/sdk/samples/cuda/vectorAdd 为例进行测试：

```
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd# pwd
/zq500/sdk/samples/cuda/vectorAdd
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd# mkdir build
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd# cd build/
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd/build# cmake ..
-- The CXX compiler identification is Clang 15.0.6
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /zq500/sdk/bin/dlcc - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Using DLI_V2
-- Configuring done
-- Generating done
-- Build files have been written to: /zq500/sdk/samples/cuda/vectorAdd/build
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd/build# make
[ 33%] Building CXX object CMakeFiles/vector_add.dir/vector_add.cpp.o
[ 66%] Building CXX object CMakeFiles/vector_add.dir/vector_add_kernel.cu.o
[100%] Linking CXX executable vector_add
[100%] Built target vector_add
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd/build# ls
CMakeCache.txt CMakeFiles Makefile cmake_install.cmake vector_add
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd/build# ./vector_add
[Vector addition of 50000 elements]
Copy input data from the host memory to the dli_cuda device
Copy output data from the dli device to the host memory
Test PASSED
(sdk) (base) root@supervisor-X7850H0:/zq500/sdk/samples/cuda/vectorAdd/build#
```

5.4 文件拷贝

5.4.1 本地和服务端之间复制文件

scp命令允许本地和服务端之间复制文件，参赛队伍的文件请放在**个人用户目录**下。

```
scp -r 本地路径 usr_01@10.110.12.10:/home/usr_name
```

以usr_01为例，在**本地终端**中输入上述命令，将本地文件传输到服务器上的个人用户目录下：

```
C:\Users\Administrator\Desktop\集训赛参赛选手说明\pictures>scp -r *.*.png usr_01@10.110.12.10:/home/usr_01
usr_01@10.110.12.10's password:
Snipaste_2026-05-09_19-18-41.png 100% 769KB 31.3KB/s 00:24
Snipaste_2026-05-09_19-19-48.png 100% 208KB 31.4KB/s 00:06
Snipaste_2026-05-09_19-21-24.png 100% 63KB 11.1KB/s 00:05
Snipaste_2026-05-09_19-22-22.png 100% 37KB 31.8KB/s 00:01
Snipaste_2026-05-09_19-26-03.png 100% 630KB 25.0KB/s 00:25
image-20260507212416560.png 100% 57KB 16.0KB/s 00:03
image-20260507212632393.png 100% 27KB 34.7KB/s 00:00
image-20260507212707579.png 100% 27KB 8.9KB/s 00:03
image-20260507212725378.png 100% 24KB 34.7KB/s 00:00
image-20260507212733355.png 100% 17KB 15.2KB/s 00:01
image-20260507212739504.png 100% 11KB 7.9KB/s 00:01
image-20260507212745642.png 100% 24KB 17.1KB/s 00:01
image-20260507224121104.png 100% 299KB 24.6KB/s 00:12
image-20260507224317322.png 100% 241KB 34.7KB/s 00:06
image-20260507225130445.png 100% 47KB 16.1KB/s 00:02
image-20260507225322601.png 100% 24KB 10.4KB/s 00:02
image-20260507225415900.png 100% 595KB 23.1KB/s 00:25
```

除了scp命令外，FinalShell等远程连接工具还提供了快捷文件传输方式，直接拖动文件到对应目录即可，注意服务器上文件应该传输到**个人用户目录**。

The screenshot shows the FinalShell 4.6.5 interface. On the left, there's a sidebar with system information (IP: 10.110.12.10, CPU: 0%, Memory: 11.6G/814.6G) and a file explorer. The main terminal window shows the output of a connection to an Ubuntu 20.04.6 LTS host. The terminal output includes welcome messages, documentation links, and information about security updates. The file explorer at the bottom shows the directory structure of the host, with a red box highlighting the /home/usr_01 directory.

5.4.2 服务器和容器之间复制文件

宿主机（即服务器）-> 容器

`sudo docker cp 宿主机路径 docker_name:/容器内路径`

```
usr_01@supervisor-X7850H0:~$ sudo docker cp ./image-20260507224121104.png gpu_01:/zq500
Successfully copied 308kB to gpu_01:/zq500
usr_01@supervisor-X7850H0:~$ sudo docker exec -it gpu_01 bash
(base) root@supervisor-X7850H0:/# cd /zq500/
(base) root@supervisor-X7850H0:/zq500# ls
cuda11      cuda_11_7_cudnn_8.5.0.patch  image-20260507224121104.png
cuda11.tar  denglin-sdk-MR-4.020101.92-202512311824-manylinux_2_28-x86_64.run  sdk
(base) root@supervisor-X7850H0:/zq500#
```

容器-> 宿主机（即服务器）

`sudo docker cp docker_name:/容器内路径 宿主机路径`